

Business Intelligence, AI und Data Wrangling

12 Aufgaben mit Python- und R-Code-Snippets

Basierend auf den Slides Week 10

26. April 2026

Viel Spass !! :-)

1 Ziel und Kompetenzraster

Dieses Aufgabenblatt verbindet die zentralen Inhalte der Slides mit praktischer Datencrunchen in Python und R. Im Fokus stehen AI-BI-Systeme, Data Wrangling, Data Cleansing, Record Matching, Schema Matching, Similarity Functions, Hidden Markov Models, Clustering, Imputation, Outlier Detection und Entscheidungsunterstützung.

Kompetenzen die man entwickeln sollte

- Datenqualität beurteilen: Missing Values, Ausreißer, Inkonsistenzen, Dubletten und Schemafehler erkennen.
- BI-Entscheidungen datenbasiert begründen: KPIs, Ursachenanalyse, Forecasting und Handlungsempfehlungen ableiten.
- Methoden vergleichen: *RegEx*, *Edit Distance*, *Jaccard*, *Cosine Similarity*, *Clustering*, *HMMs* und *AI/Embedding-Ansätze* bewerten.
- Governance berücksichtigen: Quellen, Berechnungslogik, Rollenrechte, Erklärbarkeit und menschliche Entscheidungsverantwortung einplanen.

2 Überblick über die 12 Aufgaben

Nr	Thema	Code	Kernleistung
1	Data Profiling und Qualitätsdiagnose	Python	Datenprobleme systematisch finden
2	Datum, Währung und Stornos standardisieren	Python	Korrekten Umsatz berechnen
3	KPI Semantic Layer	Python	Umsatz, Marge, Churn konsistent definieren
4	Edit Distance für Tippfehler	Python	Record Matching mit Schwellenwert

Nr	Thema	Code	Kernleistung
5	Jaccard für Produktnamen	Python	Token-basierte Ähnlichkeit vergleichen
6	Schema Matching	Python	Spalten automatisch mappen
7	RegEx Parsing vs. HMM-Denken	R	Strukturierte Extraktion begründen
8	Clustering für Deduplikation	Python	Ähnliche Kunden/Produkte gruppieren
9	Outlier Detection	Python	Anomalien mit Isolation Forest erkennen
10	Zeitreihen-Decomposition und Forecast	R	Trend/Saison/Rest interpretieren
11	Missing-Value-Imputation	Python	Mittelwert vs. modellbasierte Imputation vergleichen
12	AI-BI Decision Layer	Python	Empfehlung mit Risiko/Impact erzeugen

3 Aufgabe 1: Data Profiling - Welche Datenprobleme liegen vor?

Bezug zu den Slides: S. 22-30, Rohdatenprobleme und Data Cleansing.

Kontext. Die Slides zeigen, dass schlechte Daten zu falschen Business-Entscheidungen führen können. In dieser Aufgabe wird ein kleiner Datensatz ähnlich den Beispielen mit Episoden- und Personendaten untersucht.

Lernziele. Sie operationalisieren Datenqualitätsdimensionen wie Validität, Vollständigkeit, Konsistenz und Plausibilität. Sie berechnen Profiling-Kennzahlen und unterscheiden technische Fehler von Business-Risiken.

Auftrag. Erstellen Sie einen Data-Quality-Report. Markieren Sie ungültige Datumswerte, unrealistische Ratings, Dubletten, fehlende Werte und potenzielle Ausreißer. Formulieren Sie anschließend drei Risiken für BI-Entscheidungen.

```
import pandas as pd
```

```
episodes = pd.DataFrame({
    'id': [1,2,3,4,5,6,7],
    'title': ['live free or die', 'madrigal', 'sunset', 'grilled', 'down', 'cancer man', 'live fre or die'],
    'date':
['01.11.2012', '06.12.2013', '25.10.2011', '20.12.2009', '42.12.2009', '19.10.2008', '01.11.2012'],
    'rating': [9.3, 89, 9.3, 9.3, 8.3, 8.3, 9.3],
    'length': [43, 48, 47, 46, 333, 48, 43]
})
```

```

}))

episodes['date_parsed'] = pd.to_datetime(episodes['date'], format='%d.%m.%Y',
errors='coerce')
checks = pd.DataFrame({
    'missing_values': episodes.isna().sum(),
    'unique_values': episodes.nunique(),
    'dtype': episodes.dtypes.astype(str)
})
print(checks)
print('Invalid dates:', episodes[episodes['date_parsed'].isna()])
print('Suspicious ratings:', episodes[~episodes['rating'].between(0,10)])
print('Suspicious lengths:', episodes[~episodes['length'].between(20,90)])

```

Abgabe. Ein 1-seitiger Data-Quality-Report mit Tabelle, Priorisierung und mindestens drei konkreten Auswirkungen auf eine Managemententscheidung.

4 Aufgabe 2: Data Wrangling - Umsatz bereinigen

Bezug zu den Slides: S. 23-25.

Kontext. Umsätze kommen aus CRM, ERP und Webshop. Unterschiedliche Datumsformate, gemischte Währungen und Stornos verfälschen den Umsatz.

Lernziele. Sie implementieren ETL-Schritte nachvollziehbar, wenden Währungsumrechnung und Storno-Logik korrekt an und stellen Vorher-/Nachher-KPIs gegenüber.

Auftrag. Berechnen Sie den Umsatz einmal naiv und einmal bereinigt. Erklären Sie, wie ein scheinbares Wachstum in einen Rückgang kippen kann.

```

import pandas as pd

sales = pd.DataFrame({
    'source': ['CRM', 'ERP', 'Webshop', 'CRM', 'ERP'],
    'date': ['01/02/2026', '2026-02-03', '02/04/2026', '2026-02-05', '05/02/2026'],
    'amount': [1200, 900, 500, 2000, 700],
    'currency': ['CHF', 'EUR', 'CHF', 'EUR', 'CHF'],
    'status': ['won', 'invoice', 'sale', 'cancelled', 'invoice']
})
fx = {'CHF':1.0, 'EUR':0.95}
sales['date_iso'] = pd.to_datetime(sales['date'], errors='coerce',
dayfirst=True)
sales['amount_chf'] = sales['amount'] * sales['currency'].map(fx)
sales['signed_amount_chf'] = sales['amount_chf'].where(sales['status']!=
='cancelled', -sales['amount_chf'])
print('Naiver Umsatz:', sales['amount'].sum())

```

```
print('Bereinigter Umsatz CHF:', sales['signed_amount_chf'].sum())
print(sales[['source', 'date_iso', 'currency', 'status', 'signed_amount_chf']])
```

Abgabe. Tabelle mit naivem und bereinigtem Umsatz sowie eine Management-Interpretation: Welche Entscheidung wäre ohne Wrangling falsch?

5 Aufgabe 3: Semantic Layer - KPIs konsistent definieren

Bezug zu den Slides: S. 16-18.

Kontext. AI-BI-Systeme benötigen einen Semantic Layer, damit Kennzahlen wie Umsatz, Marge, Churn oder CAC zentral und nachvollziehbar definiert sind.

Lernziele. Sie machen KPI-Definitionen explizit, trennen Berechnungslogik von Visualisierung und AI-Agent und wenden Governance-Prinzipien auf KPIs an.

Auftrag. Implementieren Sie eine kleine KPI-Schicht als Funktionen. Diskutieren Sie, warum ein Chatbot nicht direkt rohe Tabellen interpretieren sollte.

```
import pandas as pd

df = pd.DataFrame({
    'customer_id': [1, 2, 3, 4, 5],
    'revenue': [1000, 2500, 0, 900, 1500],
    'cost': [600, 1300, 50, 700, 900],
    'active_last_month': [1, 1, 1, 1, 1],
    'active_this_month': [1, 1, 0, 1, 0]
})

def gross_margin_rate(data):
    revenue = data['revenue'].sum()
    margin = revenue - data['cost'].sum()
    return margin / revenue if revenue else None

def churn_rate(data):
    base = data['active_last_month'].sum()
    churned = ((data['active_last_month'] == 1) &
               (data['active_this_month'] == 0)).sum()
    return churned / base if base else None

print({'gross_margin_rate': gross_margin_rate(df), 'churn_rate':
      churn_rate(df)})
```

Abgabe. KPI-Dictionary mit Definition, Formel, Datenquelle und Owner. Argumentation: Welche Fehler verhindert der Semantic Layer?

6 Aufgabe 4: Record Matching mit Edit Distance

Bezug zu den Slides: S. 36, 48-59, 63-64.

Kontext. Bei Kundennamen oder Produktnamen entstehen Tippfehler und Varianten. Edit Distance misst die minimale Anzahl von Zeichenoperationen.

Lernziele. Sie verstehen Levenshtein-Distanz, wählen Schwellenwerte kritisch und bewerten False Positives sowie False Negatives.

Auftrag. Matchen Sie zwei Kundenlisten. Variieren Sie den Threshold und dokumentieren Sie die Auswirkungen.

```
def levenshtein(a, b):
    dp = [[i+j if i*j==0 else 0 for j in range(len(b)+1)] for i in
range(len(a)+1)]
    for i in range(1, len(a)+1):
        for j in range(1, len(b)+1):
            cost = 0 if a[i-1].lower() == b[j-1].lower() else 1
            dp[i][j] = min(dp[i-1][j] + 1, dp[i][j-1] + 1, dp[i-1][j-1] +
cost)
    return dp[-1][-1]

left = ['Müller AG', 'Acme Schweiz', 'Renold Analytics']
right = ['Mueller AG', 'ACME Switzerland', 'Renold Analytcs']
for l in left:
    candidates = sorted([(r, levenshtein(l, r)) for r in right], key=lambda
x: x[1])
    print(l, '->', candidates[0])
```

Abgabe. Matching-Tabelle mit Distanzwerten, begründetem Threshold und Analyse mindestens eines riskanten Matches.

7 Aufgabe 5: Jaccard-Ähnlichkeit für Produktkataloge

Bezug zu den Slides: S. 34, 44, 60-64.

Kontext. Produktbezeichnungen enthalten Tokens in unterschiedlicher Reihenfolge. Jaccard eignet sich für Mengenvergleiche, ist aber abhängig von der Tokenisierung.

Lernziele. Sie erkennen Tokenisierung als Modellierungsentscheidung, grenzen Jaccard von Edit Distance ab und bewerten Produktmatching in einem E-Commerce-Use-Case.

Auftrag. Vergleichen Sie Produktnamen aus zwei Shops. Testen Sie einfache Tokenisierung und 3-Gramme.

```
import re

def tokens(s):
```

```

    return set(re.findall(r'[a-z0-9]+', s.lower()))

def char_ngrams(s, n=3):
    s = re.sub(r'\s+', '', s.lower())
    return {s[i:i+n] for i in range(max(len(s)-n+1, 1))}

def jaccard(a, b):
    return len(a & b) / len(a | b) if a | b else 0

pairs = [('iPhone 13, 128GB, Schwarz', 'Apple iPhone 13 (128GB) - Black'),
         ('NikeAir Zoom', 'Nike Air Zoom'),
         ('Galaxy S21 Case', 'Samsung Galaxy S21 Cover')]
for a,b in pairs:
    print(a, 'VS', b)
    print('token:', round(jaccard(tokens(a), tokens(b)), 3),
          'ngram:', round(jaccard(char_ngrams(a), char_ngrams(b)), 3))

```

Abgabe. Ranking der Paare nach Ähnlichkeit und Diskussion: Wann ist Reihenfolge relevant, wann nicht?

8 Aufgabe 6: Schema Matching - Spaltennamen mappen

Bezug zu den Slides: S. 33, 37, 72.

Kontext. In Enterprise Data Warehouses heißen ähnliche Attribute oft unterschiedlich, z. B. Datum vs. SaleDate oder Kundennr vs. Kunden-ID.

Lernziele. Sie verstehen Schema Matching als Vorbedingung von Integration, wenden Ähnlichkeit auf Metadaten an und dokumentieren Mapping-Entscheidungen.

Auftrag. Erstellen Sie ein automatisches Mapping zwischen zwei Schemas. Ergänzen Sie manuelle Review-Regeln für niedrige Scores.

```

from difflib import SequenceMatcher

schema_a = ['Kundennr', 'Datum', 'Umsatz_CHF', 'Produktname']
schema_b = ['customer_id', 'SaleDate', 'RevenueCHF', 'Product_Name']
synonyms = {'kundennr': 'customer_id', 'datum': 'saledate',
            'umsatz_chf': 'revenuechf', 'produktname': 'product_name'}

def normalize(x):
    return x.lower().replace('-', '_').replace(' ', '_')

def score(a,b):
    a2 = synonyms.get(normalize(a), normalize(a))
    b2 = normalize(b)
    return SequenceMatcher(None, a2, b2).ratio()

for a in schema_a:

```

```
best = max(schema_b, key=lambda b: score(a,b))
print(a, '->', best, round(score(a,best), 2))
```

Abgabe. Mapping-Tabelle mit Score und Review-Status. Kurzer Abschnitt: Warum ist Schema Matching nicht vollständig automatisierbar?

9 Aufgabe 7: Parsing mit RegEx - Regelbasierter Ansatz?

Bezug zu den Slides: S. 46-57.

Kontext. Die Slides vergleichen HMMs und RegEx. RegEx ist stark bei festen Mustern, HMMs bei variablen Kontexten.

Lernziele. Sie extrahieren strukturierte Informationen aus Text, erkennen Grenzen regelbasierter Methoden und argumentieren, wann ein HMM oder ML-Parser sinnvoller wäre.

Auftrag. Extrahieren Sie PLZ, Stadt und Telefonnummern aus Texten. Beschreiben Sie Fälle, bei denen RegEx scheitert.

```
texts <- c(
  'Musterstr. 1, 10115 Berlin, Tel: +49 30 123456',
  'CH-4051 Basel; Telefon 061 555 44 33',
  'Kontakt: Anna Schmidt, keine Telefonnummer vorhanden'
)

extract_zip_city <- function(x) {
  m <- regmatches(x, regexpr('([0-9]{4,5})\\s+([A-Za-zÄÖÜäöüß]+)', x,
perl=TRUE))
  ifelse(length(m) == 0, NA, m)
}

extract_phone <- function(x) {
  m <- regmatches(x, regexpr('(\\+?[0-9][0-9 ]{6,})', x, perl=TRUE))
  ifelse(length(m) == 0, NA, m)
}

data.frame(text=texts, zip_city=sapply(texts, extract_zip_city),
phone=sapply(texts, extract_phone))
```

Abgabe. Extraktionstabelle und Vergleich RegEx vs. HMM in 5-7 Sätzen mit Beispielen.

10 Aufgabe 8: Clustering für Deduplikation

Bezug zu den Slides: S. 38, 43, 45, 75-82.

Kontext. Deduplikation kann als Clustering ähnlicher Entitäten formuliert werden. Im Unterrichtskontext reicht ein numerisches Feature-Set; produktiv wären Embeddings oder Siamese Networks denkbar.

Lernziele. Sie wenden Clustering als Entity-Resolution-Ansatz an, interpretieren DBSCAN-Parameter und validieren Cluster manuell.

Auftrag. Cluster Sie ähnliche Kundeneinträge anhand einfacher Textfeatures. Identifizieren Sie potenzielle Dubletten.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import DBSCAN
import pandas as pd

names = ['Amazon Inc.', 'Amazon.com', 'Amazn Incorporated', 'Müller AG',
'Mueller AG', 'Beta Health GmbH']
X = TfidfVectorizer(analyzer='char', ngram_range=(2,4)).fit_transform(names)
clusters = DBSCAN(eps=0.55, min_samples=2, metric='cosine').fit_predict(X)
print(pd.DataFrame({'name': names, 'cluster':
clusters}).sort_values('cluster'))
```

Abgabe. Cluster-Tabelle, manuelle Interpretation und Parameter-Sensitivität: Was passiert bei $\text{eps}=0.35$ und $\text{eps}=0.75$?

11 Aufgabe 9: Outlier Detection mit Isolation Forest

Bezug zu den Slides: S. 70-71, 85.

Kontext. AI-gestützte Data-Wrangling-Systeme sollen Anomalien erkennen, z. B. ungewöhnliche Transaktionen oder auffällige Messwerte.

Lernziele. Sie nutzen unüberwachtes Lernen für Datenqualität, unterscheiden Anomalien von Fehlern und priorisieren den Business Impact von Ausreißern.

Auftrag. Erkennen Sie auffällige Transaktionen. Prüfen Sie, ob es sich um Betrug, Datenfehler oder legitime Sonderfälle handeln könnte.

```
import pandas as pd
from sklearn.ensemble import IsolationForest

tx = pd.DataFrame({
    'amount':[20, 35, 42, 30, 28, 2500, 33, 31, 5000, 29],
    'hour':[10, 11, 9, 12, 10, 3, 11, 10, 2, 9],
    'foreign_transfer':[0,0,0,0,0,1,0,0,1,0]
})
```



```

})
model = IsolationForest(contamination=0.2, random_state=42)
tx['anomaly'] = model.fit_predict(tx[['amount', 'hour', 'foreign_transfer']])
tx['anomaly_score'] =
model.decision_function(tx[['amount', 'hour', 'foreign_transfer']])
print(tx.sort_values('anomaly_score'))

```

Abgabe. Tabelle mit Anomaly Score und Interpretation. Vorschlag für ein BI-Alert: Trigger, Schwelle, Empfänger, Eskalation.

12 Aufgabe 10: Zeitreihen zerlegen - Trend, Saison und Rest

Bezug zu den Slides: S. 9-11, 83.

Kontext. Die Slides zeigen saisonale Zeitreihen und die Zerlegung in Trend, saisonale und irreguläre Komponente.

Lernziele. Sie interpretieren Zeitreihenkomponenten, lesen Forecasts mit Unsicherheit kritisch und unterscheiden Saisonalität von echtem Trend.

Auftrag. Zerlegen Sie eine monatliche Zeitreihe. Beschreiben Sie, welche Komponente für eine Managemententscheidung relevant ist.

R: Zeitreihen-Decomposition mit eingebautem Datensatz AirPassengers

```

ap <- AirPassengers
fit <- decompose(ap, type = 'multiplicative')
plot(fit)

```

Einfache Prognose mit Holt-Winters

```

hw <- HoltWinters(ap)
pred <- predict(hw, n.ahead = 12, prediction.interval = TRUE)
print(head(pred))

```

Abgabe. Plot der Decomposition und Interpretation: Trend, saisonales Muster, Residuen und Konsequenzen für Planung. Vertiefung: Vergleichen Sie additive und multiplikative Decomposition.

13 Aufgabe 11: Missing-Value-Imputation - Mittelwert vs. modellbasiert

Bezug zu den Slides: S. 68-69.

Kontext. Fehlende Werte können verzerren. AI-gestützte Imputation versucht Muster in anderen Variablen zu nutzen, statt nur Mittelwerte einzusetzen.

Lernziele. Sie verstehen Missingness als methodisches Risiko, vergleichen einfache und modellbasierte Imputation und diskutieren Bias sowie Unsicherheit.

Auftrag. Simulieren Sie fehlende Werte und vergleichen Sie Mean-Imputation mit Iterative Imputer. Interpretieren Sie die Unterschiede.

```
import numpy as np
import pandas as pd
from sklearn.experimental import enable_iterative_imputer # noqa
from sklearn.impute import SimpleImputer, IterativeImputer

rng = np.random.default_rng(7)
df = pd.DataFrame({
    'age': rng.normal(45, 12, 100),
    'income': rng.normal(80000, 15000, 100),
    'spend': rng.normal(5000, 1000, 100)
})
df.loc[df['income'] > 90000, 'spend'] = np.nan

mean_imp = SimpleImputer(strategy='mean').fit_transform(df)
iter_imp = IterativeImputer(random_state=7).fit_transform(df)
print('Mean spend after simple imputation:', mean_imp[:,2].mean().round(2))
print('Mean spend after iterative imputation:',
iter_imp[:,2].mean().round(2))
```

Abgabe. Kurzer Methodenvergleich und Empfehlung: Welche Imputation wäre in einem BI-Report transparent genug?

14 Aufgabe 12: Decision Layer - Handlungsempfehlungen

Bezug zu den Slides: S. 14-21.

Kontext. Ein AI-BI-System soll nicht nur Dashboards liefern, sondern Anomalien erklären, Ursachen prüfen, Prognosen erstellen und Empfehlungen begründen.

Lernziele. Sie unterscheiden Insight, Root Cause, Recommendation und Decision Log, modellieren Priorisierung nach Impact, Risiko und Umsetzbarkeit und sichern menschliche Verantwortung im Entscheidungsprozess.

Auftrag. Entwickeln Sie eine einfache Scoring-Logik für Handlungsempfehlungen nach Umsatzimpact, Risiko und Umsetzbarkeit. Ergänzen Sie Quellen- und Begründungsfelder.

```
import pandas as pd

actions = pd.DataFrame({
    'action': ['Top-20-Deals prüfen', 'Discount-Freigabe', 'Kampagne
stoppen', 'Lagerbestand erhöhen'],
    'impact_chf': [120000, 80000, 40000, 60000],
    'risk': [0.2, 0.5, 0.3, 0.4], # 0 = gering, 1 = hoch
    'feasibility': [0.9, 0.7, 0.8, 0.6] # 0 = schwer, 1 = leicht
})
actions['decision_score'] = actions['impact_chf'] * actions['feasibility'] *
```

```
(1 - actions['risk'])  
actions = actions.sort_values('decision_score', ascending=False)  
print(actions)
```

Abgabe. Priorisierte Entscheidungsvorlage mit Score, Erklärung, Datenquellen und Verantwortlichkeit. Kurze Governance-Notiz: Warum darf die AI nicht autonom entscheiden?